

README / Setup Guide

Overview

This expense management application is designed to streamline the process of submitting, approving, and reporting on employee expenses within an organization. It provides a user-friendly interface for employees to submit expense claims and for managers to approve or reject these claims. Additionally, it offers comprehensive reporting features to analyze expense data. The core benefits include improved efficiency in expense processing, better financial oversight, and enhanced compliance with organizational policies.

Key Features

- **Expense Submission:** Employees can submit expense claims with detailed descriptions and amounts.
- **Expense Approval Workflow:** Managers can approve or reject submitted expenses, updating the status accordingly.
- **Expense Reporting:** Generate detailed reports on total expenses, approved and rejected expenses.
- **User Authentication and Authorization:** Secure access control using Spring Security with role-based access.

Prerequisites

- Java 11 or higher
- PostgreSQL 12 or higher
- Node.js 14 or higher
- npm 6 or higher
- Docker (optional for database setup)

Project Structure

The project is structured as follows:

- `backend/`: Contains the backend code, including REST controllers, service classes, repositories, and security configurations.
- `frontend/`: Contains the frontend code, built with React, for the user interface.
- `shared/`: Contains shared utilities and configurations.
- `docs/`: Contains project documentation and API references.

Installation & Setup

Backend Setup

1. Clone the repository: `git clone <url>`
2. Navigate to the project: `cd expense-management`
3. Create a virtual environment: `python -m venv venv`
4. Activate the virtual environment: `source venv/bin/activate` (Linux/Mac) or `venv\Scripts\activate` (Windows)
5. Install dependencies: `pip install -r backend/requirements.txt`
6. Set up the PostgreSQL database by running the schema script provided in the `backend/schema.sql` file.

Frontend Setup

1. Navigate to the frontend directory: `cd frontend`
2. Install dependencies: `npm install`
3. Build/prepare assets: `npm run build`

Configuration

Required Environment Variables

- `DATABASE_URL`: Controls the PostgreSQL database connection string, e.g., `postgresql://username:password@localhost:5432/expensedb`
- `JWT_SECRET`: Controls the secret key for JWT authentication, e.g., `your_jwt_secret`

Optional Environment Variables

- `PORT` (default: 5000): Controls the port on which the backend server runs.

Running the Application

Starting the Backend

1. Ensure the virtual environment is activated.
2. Set environment variables: `export DATABASE_URL="postgresql://username:password@localhost:5432/expensedb"` and `export JWT_SECRET="your_jwt_secret"`
3. Start the server: `python backend/app.py`
4. Verify: Backend is running on `http://localhost:5000`

Starting the Frontend

1. Navigate to the frontend directory: `cd frontend`
2. Start the development server: `npm run dev`
3. Open the browser: `http://localhost:3000`
4. Verify: UI loads and connects to the backend.

Verification Checklist

- ☐ Backend health check responds at `GET /api/health` with 200 OK
- ☐ Frontend loads without console errors
- ☐ Basic workflow completes end-to-end (e.g., submit an expense, approve it, and generate a report)
- ☐ API endpoints respond with expected data format

Troubleshooting

Backend Issues

- **Port already in use:** Change the `PORT` environment variable or kill the process on port 5000.
- **Module not found:** Run `pip install -r backend/requirements.txt` again.
- **Database connection error:** Verify the connection string in the config, and ensure the PostgreSQL service is running.
- **Authentication error:** Verify API keys/tokens in `.env` are correct and not expired.

Frontend Issues

- **Blank page or won't load:** Check the browser console for errors, and ensure the backend is running.
- **API connection error:** Verify the backend URL in the config, and check CORS settings.
- **Module resolution error:** Delete `node_modules` and run `npm install` again.
- **Build fails:** Ensure the Node.js version matches the requirements, and clear the cache: `npm cache clean --force`.

Common Workflows

Submit an Expense

1. Navigate to the expense submission page.
2. Fill in the expense details (employee ID, amount, description, date).
3. Click the "Submit" button.
4. Verify: Expense is submitted successfully, and a confirmation message is displayed.

Approve an Expense

1. Navigate to the expense approval page.
2. Select an expense to approve.
3. Click the "Approve" button.
4. Verify: Expense status is updated to "APPROVED".

Generate an Expense Report

1. Navigate to the expense report page.
2. Select the report date range.
3. Click the "Generate Report" button.
4. Verify: Report is generated with total expenses, approved and rejected expenses.

Deployment Notes

For production deployment, ensure the following:

- Use environment variables for sensitive configurations.
- Set up a reverse proxy (e.g., Nginx) for secure communication.
- Configure database connection pooling and scaling.
- Implement CI/CD pipelines for automated testing and deployment.

Getting Help

- Check logs: Backend logs in the console, frontend logs in browser DevTools.
- Review error messages and stack traces for clues.
- Check environment variables are set correctly.
- Refer to the API Documentation and User Guide for more details.